

Đặc tả Kỹ thuật: Ứng dụng Blog Next.js

1. Thông tin dự án

1.1. Tên dự án

NextJS Blog Learning Platform

1.2. Mô tả

Ứng dụng blog cá nhân được xây dựng bằng Next.js 16 (App Router), phục vụ mục đích đào tạo và thực hành các kiến thức Next.js từ cơ bản đến nâng cao.

1.3. Mục đích Sử dụng

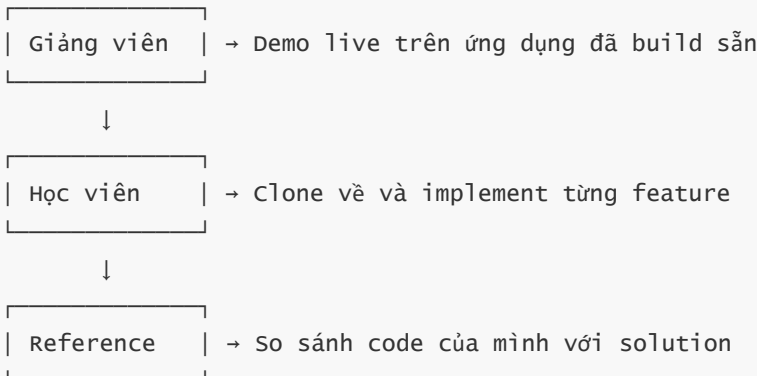
1.3.1. Demo Application cho Giảng viên

-  Demo live các concept trong lớp
-  Reference code chuẩn cho học viên
-  Troubleshooting examples
-  Deploy link: `https://nextjs-blog-demo.vercel.app` (example)

1.3.2. Learning Resource cho Học viên

-  Starter template để practice
-  Progressive commits để follow từng bước
-  Code examples cho mỗi concept
-  Exercises với solutions

1.3.3. Teaching Strategy



1.4. Mục tiêu Kỹ thuật

- Áp dụng đầy đủ các tính năng Next.js: Image, Routing, Server/Client Components, Data Fetching
- Thực hành form handling và validation
- Hiểu rõ sự khác biệt giữa SSR, CSR, SSG

- Làm việc với Advanced Routing (Nested, Dynamic, Parallel)

1.5. Clarifications

Đây là tóm tắt các quyết định đã được làm rõ để giải quyết các điểm mơ hồ trong đặc tả ban đầu:

- **Q1: Xử lý dữ liệu Contact Form?**
 - **A:** Dữ liệu từ contact form sau khi xác thực sẽ được lưu vào một file JSON cục bộ tại `data/contacts.json`.
- **Q2: Lưu trữ bình luận (Comments)?**
 - **A:** Dữ liệu bình luận sẽ được lưu vào một file JSON cục bộ tại `data/comments.json` để đảm bảo tính bền vững và chia sẻ giữa các người dùng.
- **Q3: Triển khai phân trang (Pagination)?**
 - **A:** Phân trang cho danh sách bài viết sẽ được thực hiện bằng một nút "Load More".
- **Q4: Tính toán `readingTime`?**
 - **A:** `readingTime` (thời gian đọc) sẽ được tính toán động dựa trên số lượng từ của nội dung bài viết.
- **Q5: Chức năng tìm kiếm (Search)?**
 - **A:** Thanh tìm kiếm sẽ hoạt động bằng cách gọi API để lấy danh sách bài viết đã được lọc từ server.

1.6. Terminology Guide: Teaching vs Implementation

IMPORTANT: This spec uses different terminology for teaching progression vs implementation order.

Teaching Term (Spec)	Implementation Term (Tasks)	Description
Module 1	Phase 3 (User Story 1)	Basic Setup - Homepage & Blog Listing
Module 2	Phase 4-5 (User Story 2-3)	Routing & Forms - Dynamic Routes, Contact Form
Module 3	Phase 6 (User Story 4)	Advanced Features - Search, Categories, Parallel Routes

Why the difference?

- **Modules:** Organize content for **teaching** (logical learning progression)
- **Phases:** Organize work for **implementation** (dependency-based build order)

For Students: Follow Modules when learning concepts.

For Developers: Follow Phases/Tasks when implementing features.

Cross-Reference:

- Teaching notes use "Module 1-3" (see Section 18)
-

2. Tech Stack

2.1. Core Framework

```
{
  "next": "^16.0.8",
  "react": "^19.0.0",
  "react-dom": "^19.0.0",
  "typescript": "^5.7.2"
}
```

2.2. Styling

```
{
  "tailwindcss": "^3.4.18",
  "autoprefixer": "^10.4.20",
  "postcss": "^8.4.49"
}
```

2.3. Form & Validation

```
{
  "react-hook-form": "^7.54.2",
  "zod": "^3.24.1",
  "@hookform/resolvers": "^3.9.1"
}
```

2.4. Utilities

```
{
  "date-fns": "^4.1.0",
  "react-hot-toast": "^2.4.0",
  "lucide-react": "^0.468.0",
  "clsx": "^2.1.1"
}
```

3. Cấu trúc thư mục

```
nextjs-blog/
├── app/
│   ├── (marketing)/
│   │   ├── layout.tsx
│   │   ├── page.tsx          # Homepage
│   │   ├── about/
│   │   │   └── page.tsx
│   │   └── contact/
│   │       └── page.tsx
│   └──
├── blog/
│   └── page.tsx              # Blog listing
```

```

|   |   |─ loading.tsx
|   |   |─ error.tsx
|   |   |─ [slug]/
|   |   |   |─ page.tsx          # Post detail
|   |   |   |─ loading.tsx
|   |   |   |   |─ @comments/
|   |   |   |       |─ default.tsx    # Parallel route
|   |   |   |─ category/
|   |   |       |─ [category]/
|   |   |           |─ page.tsx
|   |
|   |─ api/
|   |   |─ contact/
|   |   |   |─ route.ts          # Contact form API
|   |   |   |─ posts/
|   |   |       |─ route.ts      # Posts API (optional)
|   |
|   |─ layout.tsx              # Root layout
|   |─ loading.tsx
|   |─ error.tsx
|   |─ not-found.tsx
|   |─ globals.css
|
|─ components/
|   |─ layout/
|   |   |─ Header.tsx
|   |   |─ Footer.tsx
|   |   |─ Navigation.tsx
|   |
|   |─ blog/
|   |   |─ PostCard.tsx          # Client component
|   |   |─ PostList.tsx         # Server component
|   |   |─ PostContent.tsx
|   |   |─ CategoryBadge.tsx
|   |   |   |─ SearchBar.tsx    # Client component
|   |
|   |─ forms/
|   |   |─ ContactForm.tsx      # Client component
|   |   |   |─ CommentForm.tsx # Client component
|   |
|   |─ ui/
|   |   |─ Button.tsx
|   |   |─ Input.tsx
|   |   |─ Textarea.tsx
|   |   |─ Card.tsx
|
|─ lib/
|   |─ api.ts                  # API functions
|   |─ utils.ts                # Utility functions (calculateReadingTime,
etc.)
|   |─ validations.ts          # Zod schemas
|   |─ constants.ts
|
|─ types/
|   |─ index.ts                # TypeScript types (use 'type' not
'interface')

```

```
|
|─ data/
|   │─ contacts.json           # Contact form submissions
|   │─ comments.json         # Blog comments
|   └─ posts.json            # Blog posts (12 minimum)
|
|─ public/
|   │─ images/
|   │   │─ placeholder.jpg
|   │   └─ posts/
|   └─ favicon.ico
|
|─ .env.local
|─ .env.example               # Template with NEXT_PUBLIC_* examples
|─ .gitignore
|─ next.config.ts
|─ tailwind.config.ts
|─ tsconfig.json
|─ eslint.config.mjs
|─ postcss.config.mjs
└─ package.json
```

4. Data Models

4.1. TypeScript Types

```
// types/index.ts

/**
 * TEACHING POINT: Use 'type' for data shapes
 * - Simpler syntax
 * - Consistent with Next.js docs
 * - Use 'interface' only for extensible contracts (rare in this project)
 */

export type Post = {
  id: string
  title: string
  slug: string
  excerpt: string
  content: string
  coverImage: string
  author: Author
  category: Category
  publishedAt: string
  tags: string[]
  readingTime: number
}

export type Author = {
  id: string
  name: string
  avatar: string
```

```

    bio: string
    email?: string
  }

  export type Category = {
    id: string
    name: string
    slug: string
    description?: string
  }

  export type Comment = {
    id: string
    postId: string
    author: string
    email: string
    content: string
    createdAt: string
  }

  export type ContactFormData = {
    name: string
    email: string
    subject: string
    message: string
  }

```

4.2. Mock Data Structure

IMPORTANT: Create sufficient data for testing all features:

- **Posts:** Minimum 12 posts (to test pagination with 6 posts/page)
- **Categories:** 5 categories (Tutorial, News, Guide, Advanced, Case Study)
- **Authors:** 3 authors with different profiles
- **Comments:** 20+ comments distributed across posts
- **Contacts:** Start with empty array `[]`

```

// data/posts.json
[
  {
    "id": "1",
    "title": "Giới thiệu về Next.js 14",
    "slug": "gioi-thieu-nextjs-14",
    "excerpt": "Tìm hiểu về các tính năng mới trong Next.js 14 và App Router",
    "content": "# Nội dung bài viết...",
    "coverImage": "/images/posts/nextjs-14.jpg",
    "author": {
      "id": "1",
      "name": "Nguyễn Văn A",
      "avatar": "/images/authors/author-1.jpg",
      "bio": "Full-stack Developer"
    },
    "category": {

```

```
    "id": "1",
    "name": "Tutorial",
    "slug": "tutorial"
  },
  "publishedAt": "2024-01-15T10:00:00Z",
  "tags": ["nextjs", "react", "tutorial"],
  "readingTime": 5
}
```

5. Routing Structure

Note: This section provides the complete routing map. Detailed page implementations are in Section 7.

5.1. Public Routes

Path	Description	Type	Implementation
/	Homepage - Danh sách bài viết mới nhất	Server Component	Section 7.1
/about	Trang giới thiệu	Server Component	Phase 5 (T023a)
/contact	Form liên hệ	Client Component	Section 7.4
/blog	Danh sách tất cả bài viết	Server Component	Section 7.2
/blog/[slug]	Chi tiết bài viết	Server Component (Dynamic)	Section 7.3
/blog/category/[category]	Bài viết theo danh mục	Server Component (Dynamic)	Phase 6 (T027)

5.2. API Routes

Required API Routes (Must Implement)

Endpoint	Method	Description	Implementation
/api/contact	POST	Xử lý form liên hệ - save to data/contacts.json	Section 8.3
/api/comments	POST	Xử lý form bình luận - save to data/comments.json	Section 8.4

Endpoint	Method	Description	Implementation
<code>/api/comments</code>	GET	Lấy danh sách bình luận theo postId (for parallel route)	Section 8.4

Optional API Routes (Reference Only - Do NOT Implement)

Endpoint	Method	Why Optional	Alternative
<code>/api/posts</code>	GET	Server Components fetch directly	Use <code>getPosts()</code> from <code>lib/api.ts</code>
<code>/api/posts/[slug]</code>	GET	Server Components fetch directly	Use <code>getPostBySlug()</code> from <code>lib/api.ts</code>

Architecture Decision: This application uses **Server-First Data Fetching**. Blog post data is fetched directly in Server Components using `lib/api.ts` functions. API routes are ONLY used for:

1. **Write operations** (Contact form, Comments form)
2. **Client-side operations** (Comments GET for parallel route)

Do NOT create `/api/posts` routes - they add unnecessary complexity without benefit.

6. Component Specifications

6.1. Layout Components

Header Component

```
// components/layout/Header.tsx
'use client'

interface HeaderProps {
  // Props nếu cần
}

export default function Header({}: HeaderProps) {
  // Features:
  // - Logo
  // - Navigation menu
  // - Search bar (optional)
  // - Mobile menu toggle
}
```

Navigation Component

```
// components/layout/Navigation.tsx
'use client'

const navItems = [
  { label: 'Trang chủ', href: '/' },
  { label: 'Blog', href: '/blog' },
]
```

```

    { label: 'Giới thiệu', href: '/about' },
    { label: 'Liên hệ', href: '/contact' },
  ]

  export default function Navigation() {
    // Use Next.js Link
    // Highlight active route
    // Responsive design
  }

```

6.2. Blog Components

PostCard Component (Client Component)

```

// components/blog/PostCard.tsx
'use client'

/**
 * TEACHING POINT: Always use @/ alias for imports
 * - Configured in tsconfig.json paths
 * - Cleaner than relative imports (../../..)
 * - Easy to refactor
 */

import Image from 'next/image'
import Link from 'next/link'
import type { Post } from '@/types'

interface PostCardProps {
  post: Post
}

export default function PostCard({ post }: PostCardProps) {
  // Features:
  // - Next.js Image optimization
  // - Link to post detail
  // - Category badge
  // - Reading time
  // - Excerpt truncation
  // - Hover effects
}

```

PostList Component (Server Component)

```
// components/blog/PostList.tsx
import type { Post } from '@types'
import PostCard from '@components/blog/PostCard'

interface PostListProps {
  posts: Post[]
}

export default async function PostList({ posts }: PostListProps) {
  // Server Component
  // Render grid of PostCard
  // Handle empty state
}
```

SearchBar Component (Client Component)

```
// components/blog/SearchBar.tsx
'use client'

import { useRouter, useSearchParams } from 'next/navigation'

export default function SearchBar() {
  // Features:
  // - Real-time search
  // - Update URL params
  // - Debounce input
  // - Clear button
}
```

6.3. Form Components

ContactForm Component

```
// components/forms/ContactForm.tsx
'use client'

import { useForm } from 'react-hook-form'
import { zodResolver } from '@hookform/resolvers/zod'
import { contactSchema } from '@lib/validations'

export default function ContactForm() {
  // Features:
  // - React Hook Form
  // - Zod validation
  // - Error messages
  // - Submit to API route
  // - Success/error toast
  // - Loading state
  // - Reset form after submit
}
```

CommentForm Component

```
// components/forms/CommentForm.tsx
'use client'

interface CommentFormProps {
  postId: string
}

export default function CommentForm({ postId }: CommentFormProps) {
  // Similar to ContactForm
  // Simpler validation
  // Submit to API route (save to data/comments.json)

  /**
   * IMPLEMENTATION SCOPE:
   * - MVP: Basic submission to API, page refresh to see new comment
   * - NOT IMPLEMENTED: Optimistic updates
   *
   * To see new comments, user must refresh the page after submission.
   * This is intentional for teaching simplicity.
   */
}
```

7. Page Implementations

7.1. Homepage

```
// app/(marketing)/page.tsx
import { getPosts } from '@lib/api'
import PostList from '@components/blog/PostList'
import type { Metadata } from 'next'

export const metadata: Metadata = {
  title: 'Trang chủ - NextJS Blog',
  description: 'Blog cá nhân chia sẻ kiến thức về Next.js và React',
}

export default async function HomePage() {
  const posts = await getPosts({ limit: 6 })

  return (
    <div>
      <section className="hero">
        {/* Hero section */}
      </section>

      <section className="latest-posts">
        <h2>Bài viết mới nhất</h2>
        <PostList posts={posts} />
      </section>
    </div>
  )
}
```

```
}
```

7.2. Blog Listing Page

```
// app/blog/page.tsx
import { getPosts, getCategories } from '@lib/api'
import PostList from '@components/blog/PostList'
import SearchBar from '@components/blog/SearchBar'

interface BlogPageProps {
  searchParams: {
    q?: string
    category?: string
  }
}

export default async function BlogPage({ searchParams }: BlogPageProps) {
  const posts = await getPosts({
    search: searchParams.q,
    category: searchParams.category,
  })

  const categories = await getCategories()

  return (
    <div>
      <SearchBar />
      { /* Category filter */ }
      <PostList posts={posts} />
      { /*
        TEACHING POINT: "Load More" Pagination (Hybrid Approach)

        1. Server Component (BlogPage) fetches ALL posts initially
        2. Client Component (LoadMoreButton) handles client-side pagination state
        3. On "Load More" click, reveal next 6 posts from already-loaded data

        why Hybrid?
        - Server: SEO-friendly, all posts in initial HTML
        - Client: Interactive state management for showing/hiding posts

        Alternative (Advanced):
        - Infinite scroll with client-side fetching (not implemented in MVP)

        */ }
    </div>
  )
}
```

7.3. Blog Detail Page

```
// app/blog/[slug]/page.tsx
import { getPostBySlug, getAllPosts } from '@lib/api'
import { notFound } from 'next/navigation'
import Image from 'next/image'
```

```

import PostContent from '@/components/blog/PostContent'

interface PostPageProps {
  params: {
    slug: string
  }
}

// Generate static params for SSG
export async function generateStaticParams() {
  const posts = await getAllPosts()
  return posts.map((post) => ({
    slug: post.slug,
  })))
}

/**
 * TEACHING POINT: Dynamic Metadata with generateMetadata()
 * - Async function that runs on server
 * - Receives same params as page component
 * - Returns Metadata object
 * - Important for SEO and social sharing
 */
export async function generateMetadata({ params }: PostPageProps):
Promise<Metadata> {
  const post = await getPostBySlug(params.slug)

  if (!post) {
    return {
      title: 'Bài viết không tìm thấy',
    }
  }

  return {
    title: post.title,
    description: post.excerpt,
    openGraph: {
      title: post.title,
      description: post.excerpt,
      images: [post.coverImage],
      type: 'article',
    },
  }
}

export default async function PostPage({ params }: PostPageProps) {
  const post = await getPostBySlug(params.slug)

  if (!post) {
    notFound()
  }

  return (
    <article>
      <header>
        <h1>{post.title}</h1>

```

```

    <Image src={post.coverImage} alt={post.title} />
    { /* Author, date, reading time */ }
  </header>

  <PostContent content={post.content} />

  { /* Comments section */ }
</article>
)
}

// Revalidate every 1 hour
export const revalidate = 3600

```

7.4. Contact Page

```

// app/(marketing)/contact/page.tsx
import ContactForm from '@components/forms/ContactForm'
import type { Metadata } from 'next'

/**
 * TEACHING POINT: Static Metadata Export
 * - Next.js 13+ requires Metadata type import
 * - Export const metadata for static pages
 * - Use generateMetadata() for dynamic pages
 */
export const metadata: Metadata = {
  title: 'Liên hệ',
  description: 'Gửi tin nhắn cho chúng tôi',
}

export default function ContactPage() {
  return (
    <div className="container max-w-2xl">
      <h1>Liên hệ</h1>
      <p>Điền thông tin bên dưới để gửi tin nhắn cho chúng tôi.</p>
      <ContactForm />
    </div>
  )
}

```

8. API & Data Fetching

8.1. Utility Functions

```

// lib/utils.ts
import { clsx, type ClassValue } from 'clsx'

/**
 * TEACHING POINT: Reading Time Calculation
 * - Standard: 200 words per minute
 * - Split by whitespace to count words

```

```

    * - Round up for better UX
    */
export function calculateReadingTime(content: string): number {
    const wordsPerMinute = 200
    const wordCount = content.trim().split(/\s+/).length
    return Math.ceil(wordCount / wordsPerMinute)
}

/**
 * Utility for merging Tailwind classes
 */
export function cn(...inputs: ClassValue[]) {
    return clsx(inputs)
}

```

8.2. API Functions

```

// lib/api.ts
import { Post, Category } from '@types'
import postsData from '@data/posts.json'

export async function getPosts(options?: {
    limit?: number
    search?: string
    category?: string
}): Promise<Post[]> {
    // Simulate API delay
    await new Promise(resolve => setTimeout(resolve, 500))

    let posts = postsData as Post[]

    // Filter by search
    if (options?.search) {
        posts = posts.filter(post =>
            post.title.toLowerCase().includes(options.search!.toLowerCase())
        )
    }

    // Filter by category
    if (options?.category) {
        posts = posts.filter(post =>
            post.category.slug === options.category
        )
    }

    // Limit results
    if (options?.limit) {
        posts = posts.slice(0, options.limit)
    }

    return posts
}

export async function getPostBySlug(slug: string): Promise<Post | null> {
    await new Promise(resolve => setTimeout(resolve, 300))

```

```

    const post = postsData.find(p => p.slug === slug)
    return post || null
  }

  export async function getAllPosts(): Promise<Post[]> {
    return postsData as Post[]
  }

  export async function getCategories(): Promise<Category[]> {
    const posts = postsData as Post[]
    const categoryMap = new Map<string, Category>()

    posts.forEach(post => {
      categoryMap.set(post.category.id, post.category)
    })

    return Array.from(categoryMap.values())
  }

```

8.3. Contact API Route

```

// app/api/contact/route.ts
import { NextRequest, NextResponse } from 'next/server'
import { z } from 'zod'
import fs from 'fs/promises'
import path from 'path'

const contactSchema = z.object({
  name: z.string().min(2, 'Tên phải có ít nhất 2 ký tự'),
  email: z.string().email('Email không hợp lệ'),
  subject: z.string().min(5, 'Tiêu đề phải có ít nhất 5 ký tự'),
  message: z.string().min(10, 'Nội dung phải có ít nhất 10 ký tự'),
})

export async function POST(request: NextRequest) {
  try {
    // TEACHING POINT: Type as unknown first, then validate
    const body: unknown = await request.json()

    // TEACHING POINT: Zod validation with parse()
    const validatedData = contactSchema.parse(body)

    // TEACHING POINT: File persistence with fs.promises
    const contactsPath = path.join(process.cwd(), 'data', 'contacts.json')

    // Read existing contacts
    let contacts = []
    try {
      const fileContent = await fs.readFile(contactsPath, 'utf-8')
      contacts = JSON.parse(fileContent)
    } catch {
      // File doesn't exist yet, start with empty array
      contacts = []
    }
  }

```

```

// Add new contact with metadata
const newContact = {
  id: Date.now().toString(),
  ...validatedData,
  createdAt: new Date().toISOString(),
}
contacts.push(newContact)

// Write back to file with pretty formatting
await fs.writeFile(contactsPath, JSON.stringify(contacts, null, 2), 'utf-8')

return NextResponse.json({
  success: true,
  message: 'Cảm ơn bạn đã liên hệ! Chúng tôi sẽ phản hồi sớm nhất có thể.',
})
} catch (error) {
  if (error instanceof z.ZodError) {
    return NextResponse.json(
      { success: false, errors: error.errors },
      { status: 400 }
    )
  }

  return NextResponse.json(
    { success: false, message: 'Đã có lỗi xảy ra' },
    { status: 500 }
  )
}
}

```

8.4. Comment API Route

```

// app/api/comments/route.ts
import { NextRequest, NextResponse } from 'next/server'
import { z } from 'zod'
import fs from 'fs/promises'
import path from 'path'

const commentSchema = z.object({
  postId: z.string().min(1, 'Post ID là bắt buộc'),
  author: z.string().min(2, 'Tên phải có ít nhất 2 ký tự'),
  email: z.string().email('Email không hợp lệ'),
  content: z.string()
    .min(5, 'Bình luận phải có ít nhất 5 ký tự')
    .max(500, 'Bình luận không được quá 500 ký tự'),
})

/**
 * TEACHING POINT: Comment Persistence
 * - Similar to Contact API but for comments
 * - Appends to data/comments.json
 * - Includes postId for filtering
 */
export async function POST(request: NextRequest) {

```

```

try {
  const body: unknown = await request.json()
  const validatedData = commentSchema.parse(body)

  const commentsPath = path.join(process.cwd(), 'data', 'comments.json')

  // Read existing comments
  let comments = []
  try {
    const fileContent = await fs.readFile(commentsPath, 'utf-8')
    comments = JSON.parse(fileContent)
  } catch {
    comments = []
  }

  // Add new comment with metadata
  const newComment = {
    id: Date.now().toString(),
    ...validatedData,
    createdAt: new Date().toISOString(),
  }
  comments.push(newComment)

  // Write back to file
  await fs.writeFile(commentsPath, JSON.stringify(comments, null, 2), 'utf-8')

  return NextResponse.json({
    success: true,
    comment: newComment,
    message: 'Bình luận đã được gửi thành công!',
  })
} catch (error) {
  if (error instanceof z.ZodError) {
    return NextResponse.json(
      { success: false, errors: error.errors },
      { status: 400 }
    )
  }

  return NextResponse.json(
    { success: false, message: 'Đã có lỗi xảy ra' },
    { status: 500 }
  )
}
}

/**
 * TEACHING POINT: GET Comments by Post ID
 * - Filter comments for specific post
 * - Used by parallel route @comments
 */
export async function GET(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url)
    const postId = searchParams.get('postId')

```

```

const commentsPath = path.join(process.cwd(), 'data', 'comments.json')

let comments = []
try {
  const fileContent = await fs.readFile(commentsPath, 'utf-8')
  comments = JSON.parse(fileContent)
} catch {
  return NextResponse.json([])
}

// Filter by postId if provided
if (postId) {
  comments = comments.filter((comment: any) => comment.postId === postId)
}

// Sort by newest first
comments.sort((a: any, b: any) =>
  new Date(b.createdAt).getTime() - new Date(a.createdAt).getTime()
)

return NextResponse.json(comments)
} catch (error) {
  return NextResponse.json(
    { success: false, message: 'Đã có lỗi xảy ra' },
    { status: 500 }
  )
}
}

```

Note: This API route is **REQUIRED** for the Comment feature to work properly.

9. Validation Schemas

```

// lib/validations.ts
import { z } from 'zod'

export const contactSchema = z.object({
  name: z.string()
    .min(2, 'Tên phải có ít nhất 2 ký tự')
    .max(50, 'Tên không được quá 50 ký tự'),
  email: z.string()
    .email('Email không hợp lệ'),
  subject: z.string()
    .min(5, 'Tiêu đề phải có ít nhất 5 ký tự')
    .max(100, 'Tiêu đề không được quá 100 ký tự'),
  message: z.string()
    .min(10, 'Nội dung phải có ít nhất 10 ký tự')
    .max(1000, 'Nội dung không được quá 1000 ký tự'),
})

export const commentSchema = z.object({
  author: z.string()
    .min(2, 'Tên phải có ít nhất 2 ký tự'),
  email: z.string()

```

```
.email('Email không hợp lệ'),
content: z.string()
  .min(5, 'Bình luận phải có ít nhất 5 ký tự')
  .max(500, 'Bình luận không được quá 500 ký tự'),
})
```

10. Styling Guidelines

10.1. Tailwind Configuration

```
// tailwind.config.ts
import type { Config } from 'tailwindcss'

const config: Config = {
  content: [
    './pages/**/*.{js,ts,jsx,tsx,mdx}',
    './components/**/*.{js,ts,jsx,tsx,mdx}',
    './app/**/*.{js,ts,jsx,tsx,mdx}',
  ],
  theme: {
    extend: {
      colors: {
        primary: {
          50: '#eff6ff',
          100: '#dbeafe',
          500: '#3b82f6',
          600: '#2563eb',
          700: '#1d4ed8',
        },
      },
      fontFamily: {
        sans: ['Inter', 'sans-serif'],
      },
    },
  },
  plugins: [],
}

export default config
```

10.2. Global Styles

```
/* app/globals.css */
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  body {
    @apply bg-gray-50 text-gray-900;
  }
}
```

```
h1 {
  @apply text-4xl font-bold mb-4;
}

h2 {
  @apply text-3xl font-semibold mb-3;
}

h3 {
  @apply text-2xl font-semibold mb-2;
}

@layer components {
  .container {
    @apply max-w-7xl mx-auto px-4 sm:px-6 lg:px-8;
  }

  .btn {
    @apply px-4 py-2 rounded-md font-medium transition-colors;
  }

  .btn-primary {
    @apply bg-primary-600 text-white hover:bg-primary-700;
  }

  .card {
    @apply bg-white rounded-lg shadow-md p-6;
  }
}
```

11. Environment Variables

```
# .env.local
NEXT_PUBLIC_SITE_NAME="NextJS Blog"
NEXT_PUBLIC_SITE_URL="http://localhost:3000"
NEXT_PUBLIC_API_URL="http://localhost:3000/api"
```

12. Next.js Configuration

```
// next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  images: {
    domains: ['localhost'],
    formats: ['image/avif', 'image/webp'],
  },
  experimental: {
    // Enable if needed
  },
}

module.exports = nextConfig
```

13. Features Checklist

Phase 1: Basic Setup (Module 1)

- ☐ Khởi tạo Next.js project với TypeScript
- ☐ Cấu hình Tailwind CSS
- ☐ Tạo layout cơ bản (Header, Footer)
- ☐ Implement Homepage với mock data
- ☐ Sử dụng Next.js Image component
- ☐ CSS Modules hoặc Tailwind styling

Phase 2: Routing & Forms (Module 2)

- ☐ Setup App Router structure
- ☐ Tạo route groups: (marketing), blog
- ☐ Implement dynamic routes: [slug]
- ☐ Tạo Contact form với validation
- ☐ Phân biệt Client vs Server Components
- ☐ Static file serving (images, icons)
- ☐ Error handling & loading states

Phase 3: Advanced Features (Module 3)

- ☐ Data fetching từ API/mock data
- ☐ Implement SSR cho blog detail
- ☐ Implement CSR cho search/comments
- ☐ Generate static params cho SSG
- ☐ Nested routing: category/[category]
- ☐ Parallel routing: @comments slot

- ☐ SEO metadata động
- ☐ Revalidation strategy

Phase 4: Future Enhancements (NOT in MVP)

- ☐ Optimistic updates for comments (requires React Server Actions)
- ☐ Dark mode toggle
- ☐ Social sharing buttons
- ☐ Related posts recommendations

14. Testing Requirements

14.1. Manual Testing Checklist

- ☐ Tất cả routes hoạt động đúng
- ☐ Form validation hiển thị lỗi chính xác
- ☐ Images load và optimize đúng
- ☐ Loading states hiển thị khi fetch data
- ☐ Error pages hiển thị khi cần
- ☐ Responsive trên mobile, tablet, desktop
- ☐ Navigation active state chính xác
- ☐ Search functionality hoạt động

14.2. Performance

- ☐ Lighthouse score > 90
- ☐ Images được optimize
- ☐ Code splitting hiệu quả
- ☐ No unnecessary client-side JS

15. Deployment

15.1. Build Commands

```
npm run build
npm run start
```

15.2. Deployment Platforms

- Vercel (Recommended)
- Netlify
- Cloudflare Pages

15.3. Pre-deployment Checklist

- ☐ Remove console.logs
- ☐ Update environment variables
- ☐ Test production build locally
- ☐ Verify all API routes work
- ☐ Check meta tags và SEO

16. Learning Outcomes Map (Detailed Module-to-Phase Mapping)

IMPORTANT: This table provides detailed alignment. See Section 1.6 for quick reference.

This spec uses "Module" terminology for teaching progression, while uses "Phase" for implementation order:

Spec (Teaching)	Tasks (Implementation)	Description
Module 1	Phase 3 (User Story 1)	Basic Setup - Homepage & Listing
Module 2	Phase 4-5 (User Story 2-3)	Routing & Forms
Module 3	Phase 6 (User Story 4)	Advanced Features (Search, Parallel Routes)

Module 1 → Implementation

- ✔ Giới thiệu Next.js → Project setup, folder structure
- ✔ Image component → Next/Image trong PostCard
- ✔ StyleSheet → Tailwind CSS, globals.css

Module 2 → Implementation

- ✔ App Router → Route groups, dynamic routes
- ✔ Client/Server Components → Forms (client), Posts (server)
- ✔ Form & Validation → ContactForm, CommentForm với Zod
- ✔ Static File Serving → /public folder

Module 3 → Implementation

- ✔ Data Fetching → API functions, fetch trong Server Components
- ✔ SSR → Blog detail page
- ✔ CSR → Search, Comments
- ✔ Advanced Routing → Nested, Dynamic, Parallel routes
- ✔ generateStaticParams → Blog posts SSG

17. Next.js Best Practices Checklist

17.1. File Naming Conventions

-  **Reserved files:** `page.tsx`, `layout.tsx`, `loading.tsx`, `error.tsx`, `not-found.tsx` (all lowercase)
-  **Components:** PascalCase (e.g., `PostCard.tsx`, `SearchBar.tsx`)
-  **Route folders:** kebab-case (e.g., `blog-posts/`)
-  **Dynamic routes:** `[slug]/`, `[id]/`, `[...slug]/`
-  **Route groups:** `(marketing)/`, `(dashboard)/`
-  **Private folders:** `_components/`, `_lib/` (excluded from routing)

17.2. Import Conventions

```
//  GOOD: Always use @/ alias
import { Post } from '@types'
import { getPosts } from '@lib/api'
import PostCard from '@components/blog/PostCard'

//  BAD: Relative imports
import { Post } from '../../types'
import { getPosts } from '../lib/api'
import PostCard from './PostCard'
```

17.3. Type Safety

```
//  GOOD: Type unknown, then validate
const body: unknown = await request.json()
const validatedData = schema.parse(body)

//  BAD: Using any
const body: any = await request.json()

//  GOOD: Use type for data shapes
export type Post = { ... }

//  OK: Use interface only for extensible contracts
export interface ApiClient { ... }
```

17.4. Metadata Pattern

```
//  Static pages: Export const metadata
import type { Metadata } from 'next'

export const metadata: Metadata = {
  title: 'Page Title',
  description: 'Description',
}
```

```
// ✅ Dynamic pages: Use generateMetadata()
export async function generateMetadata({ params }): Promise<Metadata> {
  const data = await fetchData(params.id)
  return {
    title: data.title,
    description: data.description,
  }
}
```

17.5. Data Fetching Best Practices

```
// ✅ Server Components: Direct async/await
export default async function Page() {
  const data = await fetch('...', { cache: 'no-store' }) // Dynamic
  // or
  const data = await fetch('...', { next: { revalidate: 3600 } }) // ISR
  return <div>{data.title}</div>
}

// ✅ Client Components: Use hooks
'use client'
export default function SearchBar() {
  const [results, setResults] = useState([])

  useEffect(() => {
    fetch('/api/search')
      .then(res => res.json())
      .then(setResults)
  }, [])
}
```

17.6. Environment Variables

```
# .env.local

# ✅ Client-side (accessible in browser)
NEXT_PUBLIC_SITE_NAME="My Blog"
NEXT_PUBLIC_API_URL="https://api.example.com"

# ✅ Server-side only (NOT accessible in browser)
DATABASE_URL="postgresql://..."
API_SECRET_KEY="secret123"
```

```
// Usage
const siteName = process.env.NEXT_PUBLIC_SITE_NAME // ✅ works in client & server
const apiKey = process.env.API_SECRET_KEY // ✅ Server-only
```

17.7. Component Patterns

```
// ✅ Server Component (default)
export default async function PostList() {
  const posts = await getPosts()
  return <div>{posts.map(...)}</div>
}

// ✅ Client Component (interactive)
'use client'
export default function SearchBar() {
  const [query, setQuery] = useState('')
  return <input value={query} onChange={e => setQuery(e.target.value)} />
}
```

17.8. Error Handling

```
// app/blog/error.tsx
'use client'

export default function Error({
  error,
  reset,
}: {
  error: Error & { digest?: string }
  reset: () => void
}) {
  return (
    <div>
      <h2>Đã có lỗi xảy ra!</h2>
      <p>{error.message}</p>
      <button onClick={reset}>Thử lại</button>
    </div>
  )
}
```

17.9. TypeScript Configuration (tsconfig.json)

```
{
  "compilerOptions": {
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "noImplicitOverride": true,
    "paths": {
      "@/*": ["./*"]
    }
  }
}
```